JosephsonCircuitsOptimizer.jl

u ar

Des topsetup,simulationandnonlinearanalysis

Repository: `SQE-INRiM/JosephsonCircuitsOptimizer`

Project format: `jco`

Target release: `v0.5.0(inpreparation)`

Document date: `November2026`

This short guide introduces the JCO desktop application and the most common simulation workflow.

For detailed syntax, solver options, data storage and nonlinear feedback, see the complete JCO User Manual.

The instructions describe the upcoming v0.5.0 GUI; published releases may contain earlier features.

JosephsonCircuitsOptimizer.jl (JCO) connects JosephsonCircuits.jl (https://josephsoncircuits.org/) simulations with linear design-space exploration, surrogate optimization, and nonlinear harmonic-balance (HB) analysis.

ns allan laun

Download the desktop package from JCO Releases (https://github.com/S E-INRiM/JosephsonCircuitsOptimizer/releases). A Julia installation is required; the desktop packages do not bundle Julia.

- Windows: Run the JCO-GUI- version -Windows-Setup.exe installer, or launch JCO-GUI- version -Windows-Portable.exe without installation.
- Linux (x86-64): Download the .AppImage, then make it executable and run it:

```
chmod +x JCO-GUI-<version>-Linux-x64.AppImage
./JCO-GUI-<version>-Linux-x64.AppImage
```

Linux packaged support is preliminary and has not been extensively tested. It may depend on distribution-specific libraries. On Windows, unsigned research builds may trigger SmartScreen; Windows 11 Smart App Control can separately block Julia DLLs. See the warnings in the repository README before changing any security settings.

Open Julia settings in the application menu, select the Julia executable and set the number of threads. The first launch may need internet access while JCO prepares its Julia environment.

ar a r e

Choose a bundled example or New project from the menu. JCO has three main pages: Setup defines the experiment, Run performs simulations, and Results explores stored data.

Save the project as a .jco file before running. This portable file keeps user inputs and saved numerical runs together; Save, Save As, and Open manage it. ou can reopen a project later without losing its previous runs.

n ure e u

In Model & inputs, define the Julia circuit, design parameters, and sources:

- Circuit: Provide create user circuit(device params set), returning the JosephsonCircuits circuit and component definitions. Use Circuit preview to inspect the resolved model.
- Device parameters: Choose Fixed, Range (start/stop/step), or List for each parameter. Parameter names must match those used in your circuit code.
- Sources: Set each port, AC/DC type, source frequency, Linear amplitude, and Nonlinear amplitude. Available amplitude modes include Fixed, Range, List, and user-defined Parametric functions. The Julia solver receives physical currents in amperes.

In Computation, choose the probe-frequency Start/Stop (GHz) and Step (MHz), optimization iterations/samples, and harmonic/solver settings. The linear stage normally uses first-order harmonic limits; the nonlinear stage needs sufficient harmonics for the pumped response. Surrogate optimization is skipped for a fixed-only device study.

In Metrics & constraints, define the cost and performance functions. The cost is minimized during linear exploration/optimization; performance is maximized over nonlinear working points. our Julia functions define any physical masks and constraints.

A minimal example (adjust ports and the metric for your circuit):

```
function user_cost(sol, device_params_set, nonlinear_correction)
    s11 = sol.linearized.S((0,), 1, (0,), 1, :)
    metric = mean(10 .* log10.(abs2.(s11))) + nonlinear_correction
    return (metric = metric,)
end

function user_performance(sol, device_params_set, source_amps, source_freqs)
    s21 = sol.linearized.S((0,), 2, (0,), 1, :)
    gain = 10 .* log10.(abs2.(s21))
    return (performance = mean(gain),)
end
```

The sol input is a full JosephsonCircuits HB solution, not an S-parameter dictionary. Return additional named scalar metrics to store them for Results. Use save data variable name to retain selected frequency-dependent arrays.

unan e l reresul s

Choose Run to execute the full Linear - Optimization - Harmonic balance workflow. Available stage-only actions can reuse compatible previous work. Track progress and log messages while the computation runs, and check the convergence of nonlinear working points before drawing physical conclusions.

In Results, select a stored Run, then inspect its Linear heatmaps/slices, Optimization landscape/history, or HB working-point responses. Select a point to inspect saved frequency-resolved arrays. Use Export CSV for numerical data and Simulation info for stored settings; exports do not replace saving the .jco project.

naln nl near ee a

Feedback is an optional full-run feature for a varying device-parameter space. Enable Nonlinear feedback in Computation, define a reference current (A) for each source (including DC), and write a user correction function:

```
function user_nonlinear_correction(sol_linear, sol_nonlinear, device_params_set)
    s_off = sol_linear.linearized.S((0,), 1, (0,), 1, :)
    s_on  = sol_nonlinear.linearized.S((0,), 1, (0,), 1, :)
    db(x) = 10 .* log10.(abs2.(x))
    return mean(db(s_on)) - mean(db(s_off))
end
```

Both inputs are complete HB solutions. JCO passes the scalar returned by this function to user cost as nonlinear correction. our cost must explicitly use it. A uniform additive correction does not, by itself, move the optimizer minimizer.

Choose Fixed cycles for a specified number of corrections, or Metric convergence to monitor Cost, Performance, or Correction with an absolute or relative ( ) tolerance and a maximum number of cycles. Cycle 0 is the uncorrected baseline; convergence compares successive completed corrected cycles.

Results offers a cycle selector and a separate Nonlinear feedback view showing evaluated HB performance and parameter evolution. Inspect each cycle's Linear/Optimization/HB data independently.

ur er n r a n

The complete JCO User Manual (JCO Documentation.pdf) documents the full GUI, Julia function contracts, solver options, data retention, project files, and feedback convergence criteria. See the repository README (../README.md) for installation warnings, source information and citation details.

This uick Start describes features being prepared for v0.5.0. The latest published desktop build may not yet include nonlinear feedback or all of the controls described here.